



Universidad Mariano
Gálvez de Guatemala.

Componente 1 – Análisis de Sistemas 2

9989-22-5223- DYLAN ALBERTO ROLDAN PEREZ

9989-21-5704 ANTHONY JAIR ENRIQUE DE LEON DE LEON

9989-23-15707 DULCE CLAUDIA MICHELLE RAMIREZ RAC

9989-12-10925 FELIX FRANCISCO TOMA RODRIGUEZ

Índice

Introducción	3
1. Contexto del Sistema Legado.....	4
2. Diagrama del Sistema Heredado:	6
3. Matriz Comparativa de Arquitectura	7
4. Justificación técnica de la arquitectura seleccionada.....	7
Escalabilidad.....	7
Mantenibilidad.....	8
Disponibilidad	8
Seguridad	8
Conclusión.....	9

Introducción

El presente documento tiene como propósito realizar el análisis técnico del Módulo CRM heredado, con el fin de identificar su estructura actual, las principales dificultades de mantenimiento y las oportunidades de mejora. El sistema existente fue desarrollado bajo un enfoque monolítico tradicional, en el cual la presentación, las reglas del negocio y el acceso a la base de datos se encuentran estrechamente relacionados, lo que puede generar alto acoplamiento, baja cohesión y dificultades al incorporar nuevas funcionalidades.

Como parte del proceso de ingeniería inversa, se documenta la arquitectura actual mediante un diagrama UML y se identifican los principales elementos de deuda técnica presentes en el sistema. Asimismo, se realiza una comparación entre el monolito heredado y la arquitectura propuesta, evaluando sus ventajas, desventajas y riesgos.

Después del análisis se propone adoptar una arquitectura de **Monolito Modular**, utilizando MVC junto con servicios, repositorios, DTO y middleware para organizar internamente la aplicación y exponer una API REST. Esta decisión se justifica mediante los atributos de calidad de escalabilidad, mantenibilidad, disponibilidad y seguridad, buscando una solución adecuada para el alcance actual del CRM y preparada para evolucionar conforme aumenten sus necesidades.

1. Contexto del Sistema Legado

El sistema actual corresponde a un Módulo CRM desarrollado inicialmente como una aplicación monolítica en PHP, complementada con HTML, CSS y JavaScript, y conectada a una base de datos relacional. Su función principal es administrar usuarios y contactos, permitiendo registrar, consultar, modificar, buscar, activar y desactivar información. Los contactos también pueden clasificarse como generales, clientes, proveedores o empleados.

Se considera un sistema monolítico porque la interfaz, la lógica del negocio y las consultas a la base de datos se encuentran estrechamente relacionadas dentro de los mismos archivos o módulos. Al no utilizar una arquitectura como MVC, algunas páginas se encargan simultáneamente de recibir solicitudes, validar datos, ejecutar consultas y generar el contenido mostrado al usuario.

Entre sus principales puntos de dolor se encuentra el alto acoplamiento, debido a que un cambio en la base de datos o en una regla del negocio puede requerir modificaciones en diferentes partes del sistema. También presenta baja cohesión, duplicación de código y dificultad para realizar pruebas independientes. Conforme se agregan nuevas funciones, como inventarios, costos o automatizaciones, aumenta la complejidad del mantenimiento. Además, las consultas ejecutadas directamente desde distintos archivos pueden convertirse en cuellos de botella y dificultar el crecimiento y la escalabilidad del sistema.

Tecnologías Actuales

Elemento	Tecnología
Lenguaje	PHP
Framework	Ninguno
Base de datos	MySQL
Fronted	HTML, CSS, JavaScript
Infraestructura	Servidor unico

Puntos de dolor identificados

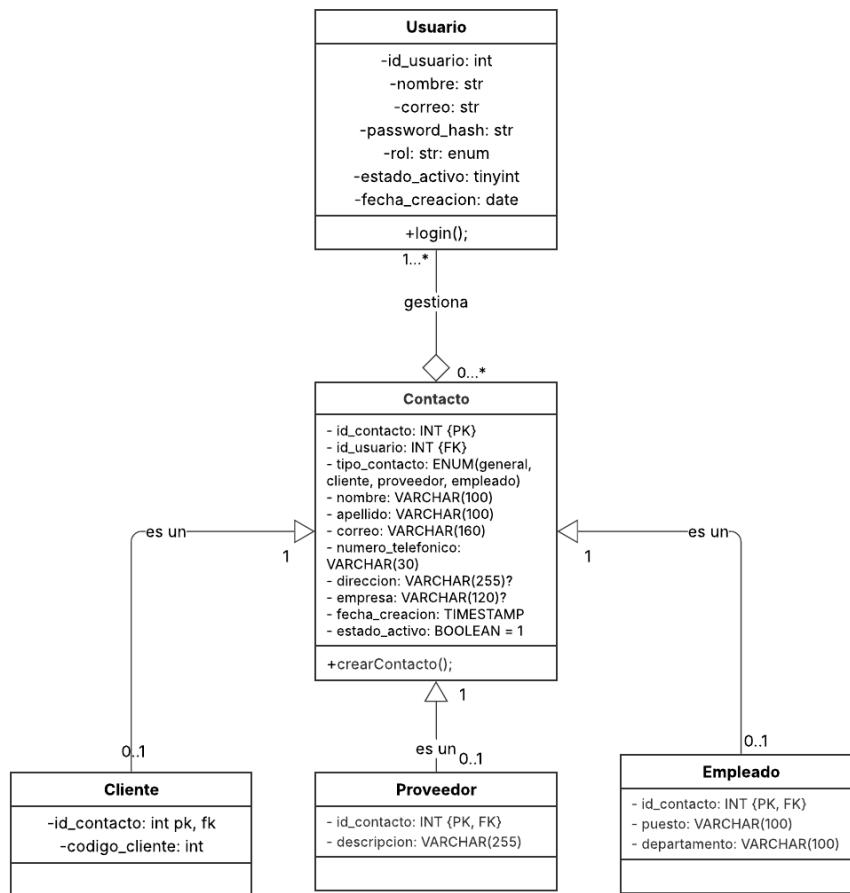
Problema	Causa	consecuencia	evidencia
acoplamiento	Logica de negocio, el acceso a datos y la presentacion estan en el mismo archivo	Un cambio en la base de datos obliga a modificar varios archivos a la vez	Archivos como contactos.php mexcaln SQL, validacion y HTML
Cuello de botella	Consultas SQL ejecutada directamente desde cada pagina	Bajo rendimiento cuando crece el volumen de contactos	Consultas repetidas de SELECT*FROM

	sin capa de repositorio		contactos en multiples visitas
Problemas de escalabilidad	Arquitectura monolitica sin separacion de responsabilidades	No se puede escalar un modulo sin escalar todo el sistema	No existen modulos independientes desplegables
Problemas de mantenibilidad	Duplica de codigo y ausencia de patron MVC	Corregir un error puede requerir cambios en varios archivos	Validaciones de formulario repetidas en distintos archivos
Problemas de disponibilidad	Un solo punto de falla	Un error no controlado puede caer toda la aplicacion	No hay manejo centralizado de errores ni monitoreo de salud
Riesgo de seguridad	Falta de capa de autenticacion centralizada: posibles consultas SQL no para metrizadas	Riesgo de inyeccion SQL y acceso no autorizado a datos	Contraseñas gestionadas sin evidencia de middleware de autorizacion
Dependencias entre componentes	Cliente, proveedor y empleado dependen directamente de la escritura interna de contacto	Un cambio en contacto puede romper las tres sibclases a la vez	Relacion de herencia rigida en el diagrama de clases actual

2. Diagrama del Sistema Heredado:

Modulo Gestion de Contactos

DYLAN ALBERTO ROLDAN PEREZ | September 3, 2026



3. Matriz Comparativa de Arquitectura

Evaluación	Sistema actual: monolito heredado	Arquitectura propuesta: monolito modular
Pros	Un solo despliegue; infraestructura sencilla; bajo costo inicial; acceso directo y rápido a la base de datos; funcionamiento suficiente para operaciones básicas del CRM.	Mantiene un solo despliegue; divide el sistema en módulos organizados; reduce el acoplamiento; mejora la mantenibilidad; facilita las pruebas; centraliza la seguridad y el acceso a datos; permite integrar una API y Redis; puede evolucionar gradualmente.
Contras	Alto acoplamiento; baja cohesión; mezcla de presentación, lógica y consultas SQL; posible duplicación de código; dificultad para realizar pruebas; mantenimiento complicado; poca flexibilidad para agregar nuevas funciones.	Requiere mayor planificación inicial; los módulos comparten el mismo proceso y despliegue; no permite escalar cada módulo de manera independiente; necesita disciplina para respetar los límites entre módulos.
Riesgos	Una modificación puede afectar varias funciones; una falla puede detener toda la aplicación; aumento progresivo de la deuda técnica; mayor dificultad para corregir vulnerabilidades; posibles cuellos de botella en las consultas directas.	Puede regresar a un monolito desorganizado si los módulos se comunican directamente; una falla crítica todavía podría afectar toda la aplicación; crecimiento excesivo de la base de datos compartida; dependencia incorrecta entre módulos si no se aplican reglas arquitectónicas.

Analisis de alternativas y Matriz comparativa

4. Justificación técnica de la arquitectura seleccionada

Para exponer la nueva API del Módulo CRM se seleccionó una arquitectura de **Monolito Modular**, implementada internamente mediante MVC y capas de servicios, repositorios, DTO y middleware. Esta decisión permite mantener una sola aplicación y un único despliegue, pero separando las funcionalidades en módulos como autenticación, usuarios, contactos, clientes, proveedores y empleados.

Escalabilidad

Aunque el Monolito Modular se despliega como una sola unidad y no permite escalar cada módulo de forma independiente, ofrece una capacidad de crecimiento suficiente para el alcance actual del CRM. La API podrá ejecutar varias instancias de la aplicación detrás de un balanceador de carga si aumenta la cantidad de solicitudes.

También se utilizarán paginación, índices en la base de datos y Redis mediante el patrón Cache-Aside para reducir consultas repetidas. La autenticación mediante JWT contribuirá a que las instancias de la API no dependan de sesiones almacenadas localmente. Además, la

separación modular facilitará extraer un módulo y convertirlo en microservicio si en el futuro presenta una carga considerablemente mayor.

Mantenibilidad

La arquitectura propuesta mejora la mantenibilidad porque establece responsabilidades específicas para cada elemento. Las rutas definirán los endpoints; los controladores recibirán las solicitudes; los servicios contendrán las reglas del negocio; los repositorios administrarán el acceso a los datos; y los DTO establecerán la información que la API puede recibir y devolver.

El flujo general será:

Ruta → Controlador → Servicio → Repositorio → Modelo → Base de datos

Esta separación reduce el acoplamiento y aumenta la cohesión. Por ejemplo, una modificación en las reglas para registrar un cliente podrá realizarse dentro del módulo correspondiente sin alterar innecesariamente la autenticación o la gestión de proveedores. También facilitará la creación de pruebas unitarias y de integración, evitando controladores con demasiadas responsabilidades.

Disponibilidad

Al tratarse de una sola aplicación, una falla crítica podría afectar toda la API. Para reducir este riesgo se propone utilizar manejo centralizado de excepciones, registros de errores, verificaciones de salud, copias de seguridad, pruebas antes de cada despliegue y un procedimiento de reversión hacia una versión estable.

En caso de crecimiento, podrán ejecutarse varias instancias de la aplicación para evitar que el servicio dependa de un único proceso. Redis funcionará únicamente como almacenamiento temporal y MySQL continuará siendo la fuente oficial de información. Por lo tanto, si Redis deja de estar disponible, la API podrá consultar directamente la base de datos, posiblemente con un menor rendimiento, pero sin perder información ni interrumpir las operaciones principales.

Seguridad

La seguridad se centralizará en la API mediante autenticación con JWT y el esquema bearerAuth. Los endpoints protegidos utilizarán middleware de autenticación y

autorización basada en roles, diferenciando las acciones permitidas para usuarios estándar y administradores.

Las contraseñas se almacenarán mediante funciones de hash seguras y nunca serán expuestas en las respuestas. Los DTO evitarán devolver campos internos como `password_hash`. También se aplicará validación de datos, consultas parametrizadas mediante el ORM, control de acceso, limitación de solicitudes y comunicación por HTTPS. Las credenciales y claves privadas deberán almacenarse en variables de entorno y no directamente en el código fuente.

Conclusión

El Monolito Modular representa la alternativa más adecuada para el Módulo CRM porque ofrece un equilibrio entre escalabilidad, mantenibilidad, disponibilidad, seguridad y complejidad. Corrige los principales problemas del monolito heredado, como el alto acoplamiento y la mezcla de responsabilidades, sin introducir los costos operativos de una arquitectura de microservicios.

MVC organizará internamente la aplicación, mientras que los módulos, servicios, repositorios, DTO y middleware permitirán exponer una API ordenada, segura y preparada para crecer. De esta manera, se conserva la sencillez de administrar una sola aplicación y se establece una base arquitectónica que podrá evolucionar si las necesidades del sistema aumentan.